

The Python Libraries that are Made for Machine Learning

This article is a guide on Python libraries for machine learning. Various libraries as well as their features are described to help readers understand them better, and enable them to then explore more advanced features.

Python is an excellent environment for building any machine learning (ML) application or system. ML involves data analysis, data engineering, feature extraction and building types of features, while Python has built-in capabilities for data analysis and data engineering. In addition, Python is very popular and is one of the *de-facto* programming languages as compared to Ruby, Perl or R. As Python provides the following advantages, one can definitely opt for it when building any ML based models or applications:

- It provides good libraries for data analysis.
- It provides an interactive computing feature including IPython and Jupyter Notebooks.
- It provides data visualisation features with multiple options to choose from.
- It provides a variety of libraries, packages and tools for data analysis.
- Python has good support for Pandas, Numpy, Matplotlib, SciPy and Scikit-learn libraries that are among the most used in ML.
- Python also provides scientific computing features including support for popular languages like C, C++ and FORTRAN.
- Python has inbuilt capabilities for doing research and prototyping.
- As Python supports multiple

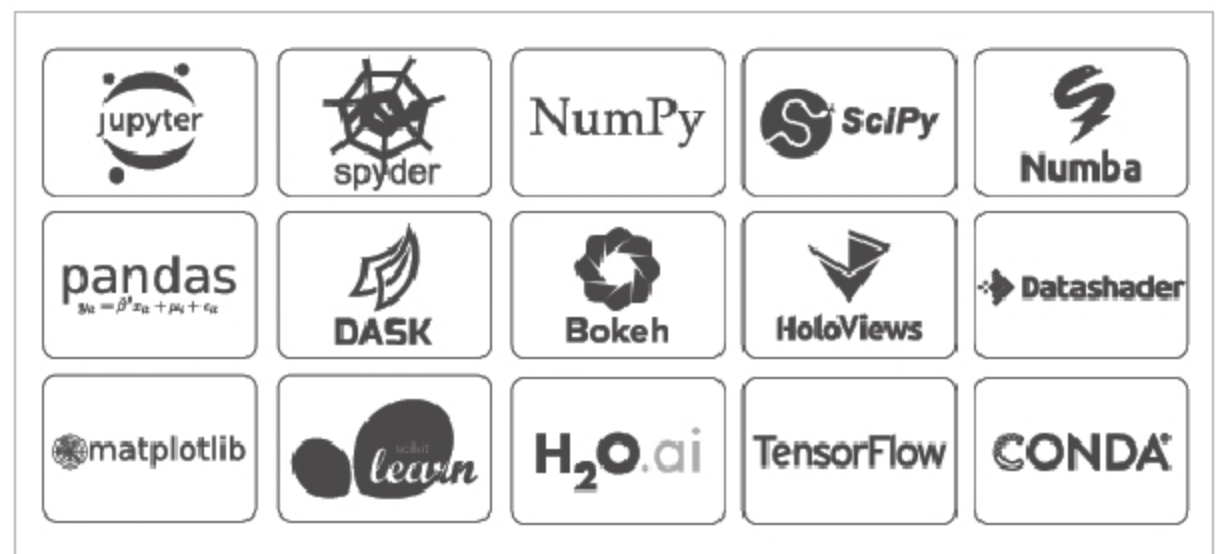


Figure 1: Anaconda distribution-supported features-libraries

languages, building production environments is very easy.

In this article, I will start with introducing the various Python libraries, one by one. This will help in better understanding how ML models/applications can be built using these. In addition, where possible, I will give examples with small code snippets. I have used the Anaconda distribution for Windows (<https://www.anaconda.com/distribution/>) for all my code examples here. Anaconda has inbuilt libraries and packages in its downloadable version, as shown in Figures 1 and 2.

```
(base) C:\Users\>jupyter notebook --version
5.6.0

(base) C:\Users\>python --version
Python 3.5.6 :: Anaconda custom (64-bit)

(base) C:\Users\>
```

Figure 2: Jupyter Anaconda version

NumPy

NumPy is the short form for Numerical Python, which is used for all numerical based calculations. By default, it provides data structures, algorithms and libraries for many of the applications involving numerical data and calculations in Python.

NumPy has a number of advantages as listed below:

- It is very fast and efficient as its engine is built using C code.
- It supports multi-dimensional arrays (using *ndarray* object).
- It has built-in functions to perform various mathematical operations and computations for arrays.
- All array based operations can be done very efficiently.
- It supports array operations like reading and writing data from disk, files and data repos.